

업무를 바꾸는 바이브 코딩 — Claude Code 직원 교육 교안 (6회차)

강의 · 삼육대학교 기획처 정진수 과장

작성일 · 2026년 10월 6일

과정 개요

6회차(회당 30-40분)를 마치면 수강생은 자기 업무에 필요한 작은 소프트웨어를 직접 기획하고, Claude Code로 만들고, 필요하면 인터넷에 배포할 수 있습니다. 핵심 메시지는 하나입니다: "기획하는 사람이 곧 만드는 사람이 된다."

- 과정명(안):** 업무를 바꾸는 바이브 코딩 — Claude Code로 내 업무 도구 직접 만들기
- 대상:** 삼육대학교 산학협력단 직원 (코딩 경험 없음 가정)
- 강의자:** 삼육대학교 기획처 정진수 과장
- 방식:** 회차마다 개념 10분 + 시연 10분 + 실습 15-20분. 실습 결과물은 매 회차 "내 컴퓨터에 남는 것" 하나로 정리

회차별 로드맵

회차	주제	실습 결과물	주요 도구
1	바이브 코딩에서 에이전틱 엔지니어링으로	Claude Code 설치, 내 폴더 파악 보고서	Claude Code (데스크톱 앱)
2	인스턴트 소프트웨어: 쓰고 지우는 내 업무 도구	엑셀 취합기, 파일 일괄 정리, 한 장짜리 HTML 도구	Claude Code, 플랜 모드, CLAUDE.md
3	Git과 GitHub: AI와 일할 때의 안전망	커밋 이력이 있는 내 프로젝트, GitHub 저장소	Git, GitHub
4	웹앱으로 키우고 Vercel로 배포하기	동료에게 링크로 공유하는 웹 도구	Next.js, Vercel
5	데이터베이스 붙이기: 접수·신청형 업무 앱	행사 신청·접수 관리 앱	Supabase (또는 Neon)
6	AI API로 지능형 업무 도구 만들기 + 안전하게 쓰기	공문·회의록 요약/분류 기능	Claude API, 환경변수

1-2회차는 로컬(내 PC), 3회차는 안전망, 4-6회차는 서버와 클라우드로 단계가 올라갑니다. 4-6회차는 시간이 빠듯하므로 강사가 미리 만든 시작 템플릿에서 출발하는 것을 권장합니다.

사전 준비물 (수강생 공지용)

- ☐ 노트북 (Windows 10 이상 또는 macOS), 관리자 권한 확인
- ☐ Claude 유료 플랜 계정 (Pro 이상) — Claude Code 사용에 필요. 기관 단체 구매 여부는 산학협력단과 사전 협의
- ☐ GitHub 계정 (3회차 전까지)
- ☐ Vercel·Supabase 계정 (4회차 전까지, GitHub 계정으로 가입)
- ☐ 실습용 샘플 파일 폴더 (개인정보가 없는 가상 데이터, 강사 배포)

1회차 · 바이브 코딩에서 에이전틱 엔지니어링으로 (35분)

이번 회차의 한 문장: "이제 소프트웨어는 사는 것이 아니라, 말로 주문해서 쓰고 버리는 것이 된다." 개념을 잡고, Claude Code를 설치해 내 폴더를 AI에게 처음 맡겨 봅니다.

학습 목표

- 바이브 코딩과 에이전틱 엔지니어링의 차이를 내 말로 설명할 수 있다
- 챗봇(웹 채팅)과 에이전트(Claude Code)의 차이를 안다
- Claude Code를 설치하고 내 폴더에서 첫 작업을 시킨다

진행표

시간	내용	방식
0-3분	여는 질문: "이런 프로그램 있으면 좋겠다" 생각해 본 적?	문답
3-13분	바이브 코딩 → 에이전틱 엔지니어링, 인스턴트 소프트웨어	강의
13-18분	챗봇 vs 에이전트: 강사 시연(같은 요청을 두 방식으로)	시연
18-23분	Claude Code 설치·로그인·화면 둘러보기	따라하기
23-33분	실습 1-1 \~ 1-3	실습
33-35분	정리, 과제 안내	강의

강의 내용

① 바이브 코딩의 탄생 (2025년 2월)

- OpenAI 공동창업자 안드레이 카파시가 2025년 2월 2일 X에 처음 쓴 말. 코드가 있다는 사실조차 잊고 "분위기(vibe)에 몸을 맡기는" 코딩 (원문)
- 연말에는 콜린스 사전 올해의 단어(2025)로 선정 (Wikipedia)
- 의미: 코드를 모르는 사람도 말로 설명해서 소프트웨어를 만들 수 있게 됐다

② 그런데 바이브만으로는 사고가 난다

- 2025년 7월, 한 AI 코딩 서비스가 "변경 금지" 지시에도 사용자의 운영 데이터베이스를 삭제
- 2025년 5월, 한 바이브 코딩 플랫폼으로 만든 웹앱 1,645개 중 170개에서 개인정보 노출 취약점 발견 (Wikipedia)
- 교훈: 만드는 것은 AI가 해도, 무엇을 만들지 정하고 결과를 확인하는 것은 사람의 몫

③ 에이전틱 엔지니어링 (2026)

- 카파시는 2026년 4월 세콰이아 어센트 등에서 "바이브 코딩은 바닥을 올리고, 에이전틱 엔지니어링은 천장을 올린다"고 정리 (요약 기사)
- 에이전틱 엔지니어링 = AI 에이전트가 계획·작성·테스트를 하고, 사람은 **요구사항 정의 → 계획 검토 → 결과 검증**을 맡는 일하는 방식
- 직원 관점 번역: 우리가 이미 잘하는 일입니다. 기획서 쓰기, 업무 지시하기, 결과물 검토하기. **행정 역량이 곧 개발 역량이 되는 시대**

구분	바이브 코딩	에이전틱 엔지니어링
태도	"알아서 해줘"	"이렇게 해줘, 그리고 확인할게"
시작	바로 만들기	계획 먼저 보기 (플랜 모드)
안전장치	없음	버전 관리(Git), 테스트, 권한 승인
어울리는 일	나만 쓰는 일회용 도구	동료가 쓰는 도구, 배포하는 서비스

이 과정의 1~2회차는 바이브 코딩으로 가볍게, 3회차부터는 에이전틱 엔지니어링의 안전장치를 하나씩 더해 갑니다.

④ 인스턴트 소프트웨어의 시대

- 예전: 프로그램 하나 = 기획서 → 예산 → 외주/전산팀 → 수개월. "이 정도 일로 프로그램을 만들까?" 하는 일은 그냥 수작업
- 지금: 말 몇 마디로 10분 만에 만들고, 쓰고, 지운다
- a16z의 아니시 아차르야(2025.8): 소프트웨어 제작을 가로막던 것은 투자 대비 효과였지만, 이제는 상상력만 남았다 (Disposable Software)
- 산학협력단 예시: 과제별 집행 엑셀 50개 취합, 증빙 파일명 일괄 변경, 협약서 만료일 추출, 행사 참가자 명찰 생성 — "프로그램까지 만들 일은 아니던" 일들

⑤ 챗봇과 에이전트는 무엇이 다른가

- 챗봇(웹 채팅): 내가 파일을 올리고, 답을 복사해서, 내가 저장하고 실행한다. 사람이 손발
- 에이전트(Claude Code): 내 컴퓨터의 폴더를 직접 열어보고, 파일을 만들고 고치고, 프로그램을 실행해 결과를 확인한 뒤 다시 고친다. 사람은 지시하고 승인한다
- 에이전트는 중요한 행동(파일 수정, 명령 실행) 전에 권한을 묻는다. 이 승인 버튼이 우리의 첫 번째 안전장치
- Anthropic 사내에서도 법무팀은 담당 변호사 안내 도구를, 마케팅팀은 광고 수백 건을 분석·생성하는 도구를 개발자 없이 만들었다 (How Anthropic teams use Claude Code)

⑥ 기획과 개발이 한 사람 안에서

- 기존: 기획자가 문서로 설명 → 개발자가 해석 → 결과물이 생각과 다름 → 재수정. 번역 손실이 크다
- 지금: 업무를 가장 잘 아는 사람이 직접 만들고, 쓰면서 바로 고친다. 기획서가 곧 설계도다

Claude Code 설치 (따라하기)

1. 권장: 데스크톱 앱 — claude.com/download에서 설치 후 로그인, 상단의 Code 탭에서 작업 폴더 선택. 터미널을 쓸 필요가 없어 비개발자 교육에 적합
2. 참고: 터미널 설치 — macOS는 `curl -fsSL https://claude.ai/install.sh | bash`, Windows PowerShell은 `irm https://claude.ai/install.ps1 | iex` 입력 후 `claude` 실행 (공식 설치 문서)
3. Pro 이상 유료 계정이 필요하다 (무료 플랜은 Claude Code 미포함)
4. Windows는 Git for Windows 설치를 권장 (3회차에도 필요)

실습

강사가 배포한 실습폴더 (가상 협약서 PDF, 집행내역 엑셀, 사진, 한글 문서가 뒤섞인 폴더)를 열고 아래 문장을 그대로 입력합니다.

1. 실습 1-1 파악하기: 이 폴더에 어떤 파일들이 있는지 종류별로 표로 정리해줘. 각 파일이 무슨 내용인지도 한 줄씩.

2. 실습 1-2 계획만 세우기: 이 파일들을 업무 유형별 폴더로 정리하려고 해. 아직 실행하지 말고 어떻게 나눌지 계획만 보여줘. → 계획을 읽고 수정 요청한 뒤 승인

3. 실습 1-3 만들기: 방금 정리한 내용을 '폴더현황보고.md' 파일로 저장해줘. → 파일 탐색기에서 새 파일이 생긴 것을 눈으로 확인
포인트는 1-2입니다. "바로 시키지 말고 계획부터"가 에이전틱 엔지니어링의 첫 습관입니다.

정리와 과제

- 핵심: AI는 만들고, 사람은 정의하고 검증한다. Claude Code는 내 파일을 직접 다루는 동료다
- 과제: 내 업무 중 반복되는 일 3가지를 적어 오기 (언제, 얼마나 자주, 몇 분 걸리는지). 2회차 실습 재료가 됩니다

강사 노트

- 실제 업무 파일(개인정보·연구비 내역)은 실습에 쓰지 않도록 처음부터 강조. 반드시 가상 데이터로
- 권한 요청 창이 뜨면 "무엇을 하려는지 읽고 누르기"를 습관화
- 예상 질문 — "챗GPT랑 뭐가 다르냐": 답을 주는 것과 일을 끝내는 것의 차이. ①⑥의 시연으로 보여주기
- 예상 질문 — "학교 문서가 밖으로 나가는 것 아니냐": 6회차 보안 파트에서 자세히 다루되, 여기서는 "대화 내용은 AI 모델로 전송된다"는 사실을 솔직하게 알리기

2회차 · 인스턴트 소프트웨어: 쓰고 지우는 내 업무 도구 (40분)

이번 회차의 한 문장: "기획이 곧 프롬프트이고, 프롬프트가 곧 프로그램이다." 내 PC에서 엑셀 취합기와 파일 정리 도구를 10분 안에 만들어 쓰고, 필요 없으면 지웁니다.

학습 목표

- 탐색 → 계획 → 실행 → 검증 4단계로 Claude Code에 일을 시킨다
- 업무 프롬프트 5요소(목적·입력·출력·조건·검증)로 요청을 쓴다
- CLAUDE.md로 "우리 팀 규칙"을 AI에게 알려 둔다

진행표

시간	내용	방식
0-5분	과제 공유: 내 업무의 반복 작업 3가지	문답
5-13분	4단계 작업법, 프롬프트 5요소, CLAUDE.md	강의
13-25분	실습 2-1 엑셀 취합기 (강사와 함께)	따라하기
25-35분	실습 2-2 또는 2-3 선택	개별 실습
35-40분	결과 공유, "저장할까 지울까" 판단	토의

강의 내용

① 4단계 작업법 (Anthropic 공식 권장 방식)

Anthropic은 "바로 만들게 하지 말고 탐색과 계획을 분리하라"고 권합니다. 그래야 엉뚱한 문제를 푸는 일을 막습니다 (Best practices for Claude Code).

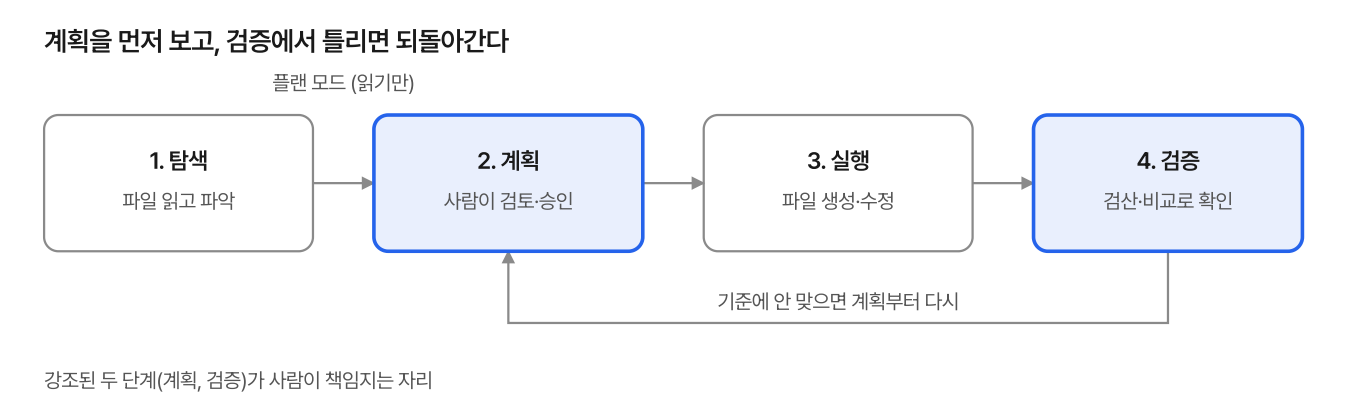


그림 1. 4단계 작업법

- 플랜 모드:** 파일을 읽기만 하고 고치지 않는 모드. 터미널에서는 `Shift+Tab`, 데스크톱 앱에서는 입력창의 모드 메뉴에서 선택
- 한 문장으로 설명되는 작은 일(오타자 수정 등)은 계획 없이 바로 시켜도 된다
- 검증이 핵심:** "합계가 원본과 맞는지 검산해서 보여줘"처럼 AI가 스스로 확인할 기준을 주면, 틀린 부분을 스스로 고친다

② 업무 프롬프트 5요소 — 기안문 쓰듯이

요소	질문	예시
목적	왜 하는가?	월말 과제별 집행을 보고용
입력	무엇을 가지고?	집행내역 폴더의 엑셀 10개
출력	무엇을 만들어?	과제별 합계 표 1개, 엑셀 파일
조건	지켜야 할 것은?	원본은 절대 수정 금지, 금액은 원 단위 쉼표
검증	맞는지 어떻게 아나?	파일별 합계와 총합을 대조해 보여줄 것

큰 작업은 거꾸로 AI에게 질문을 받습니다: 나는 [만들고 싶은 것]을 만들고 싶어. 내가 미처 생각 못한 부분까지 질문해서 요구사항을 정리한 뒤 기획서.md로 저장해줘. 기획자가 현업 담당자를 인터뷰하던 일을 AI가 나에게 해 주는 셈입니다.

③ CLAUDE.md — AI에게 주는 업무 인수인계서

폴더에 CLAUDE.md 파일을 두면 Claude가 대화를 시작할 때마다 읽습니다. /init 을 입력하면 초안을 만들어 줍니다. 짧게, 매번 필요한 것만 씁니다.

```
# 산학협력단 업무 규칙
- 모든 답변과 파일은 한국어로
- 원본 파일은 절대 수정하지 말고, 결과물은 '결과' 폴더에 새로 만들 것
- 날짜는 YYYY-MM-DD, 금액은 원 단위 천 단위 쉼표
- 파일을 삭제하거나 이동하기 전에는 변경 목록을 먼저 보여줄 것
```

④ 자주 쓰는 조작 4가지

- Esc : 작업 중 멈추기 (대화는 유지)
- Esc 두 번 또는 /rewind : 이전 시점으로 되돌리기 (파일까지)
- /clear : 주제가 바뀔 때 대화 초기화. 같은 실수를 두 번 이상 고쳤다면 새로 시작하는 편이 빠르다
- @파일명 : 특정 파일을 직접 가리키기. 스크린샷은 끌어다 놓으면 된다

실습

1. 실습 2-1 엑셀 취합기 (필수, 함께): 가상 과제 10개의 집행내역 엑셀을 하나로 모읍니다. - 플랜 모드에서: 집행내역 폴더의 엑셀들을 살펴보고, 과제별·비목별 합계를 한 장으로 모으는 방법을 계획해줘. 파일마다 양식이 조금씩 다르다면 알려줘. - 계획 승인 후: 결과/집행현황_취합.xlsx로 만들고, 파일별 합계와 총합이 맞는지 계산 결과를 보여줘. - 확장: 매달 다시 쓸 수 있게, 폴더만 바꾸면 돌아가는 더블클릭 실행 파일로 만들어줘.
2. 실습 2-2 증빙 파일명 일괄 변경 (선택): 증빙 폴더 파일명을 '날짜_과제번호_품목' 규칙으로 바꾸고 싶어. 먼저 변경 전/후 표만 보여주고, 내가 승인하면 바꿔줘. 되돌리기 방법도 만들어 줘.
3. 실습 2-3 한 장짜리 HTML 도구 (선택): 출장지와 일수를 넣으면 우리 학교 여비 기준으로 출장비를 계산하는 화면을 HTML 파일 하나로 만들어줘. 더블클릭하면 브라우저에서 열리게. (여비 기준은 가상 값으로 제공)
4. 내 과제에 적용 (시간 남으면): 1회차 과제에서 하나를 골라 "질문해서 기획서로 정리해줘" 프롬프트로 시작

정리: 저장할까, 지울까

- 한 번 쓰고 끝날 도구 → 지운다. 다시 필요하면 다시 만들면 된다. 남겨야 할 것은 도구가 아니라 **프롬프트와 기획서**
- 매달 또는 동료와 같이 쓸 도구 → 버전 관리가 필요하다. 3회차 Git으로 이어집니다

강사 노트

- 엑셀 처리에 Python이 필요할 수 있음. Claude가 설치를 시도하지만, 학교 PC 보안 정책으로 막힐 수 있으니 **사전에 실습 PC 한 대로 리허설 필수**
- 한글(HWP) 파일은 직접 다루기 어렵다. HWPX·PDF·Word로 변환해 쓰도록 안내
- 결과가 사람마다 다르게 나오는 것이 정상. "정답 코드"가 아니라 "원하는 결과"를 확인하는 것이 평가 기준
- 선택 실습 덕분에 빠른 사람과 느린 사람의 격차를 흡수할 수 있다

3회차 · Git과 GitHub: AI와 일할 때의 안전망 (35분)

이번 회차의 한 문장: "Git은 게임의 세이브 포인트, GitHub는 그 세이브를 올려두는 클라우드다." 명령어는 외울 필요 없습니다. 개념 4개만 알고, 실행은 Claude Code에게 말로 시킵니다.

학습 목표

- 저장소·커밋·푸시·.gitignore 4가지 개념을 설명할 수 있다
- Claude Code에게 "커밋해줘", "어제 상태로 되돌려줘"를 시켜 본다
- 2회차에 만든 도구를 GitHub 비공개 저장소에 올린다 (4회차 배포 준비)

진행표

시간	내용	방식
0-3분	"최종_진짜최종_v3.hwp" 경험 나누기	문답
3-13분	왜 버전 관리인가, 핵심 개념 4가지	강의
13-18분	AI가 망친 코드를 커밋으로 되살리는 시연	시연
18-32분	실습 3-1\~ 3-3	실습
32-35분	정리	강의

강의 내용

① 왜 AI와 일할수록 버전 관리가 필수인가

- AI는 한 번에 여러 파일을 빠르게 고친다. 잘되던 기능이 다른 요청 때문에 깨지는 일이 흔하다
- Claude Code에도 되돌리기(/rewind)가 있지만, 공식 문서도 "Git을 대신하지 않는다"고 밝힌다 (Best practices)
- 바이브 코딩과 에이전틱 엔지니어링을 가르는 첫 안전장치가 바로 이것

② 핵심 개념 4가지 — 행정 업무로 비유하면

개념	뜻	행정 비유
저장소 (repository)	변경 이력을 기록하는 폴더	문서대장이 붙은 서류철
커밋 (commit)	지금 상태를 메모와 함께 저장하는 세이브 포인트	결재 버전 + 변경 사유
푸시 (push)	내 PC의 이력을 GitHub에 올리기	기록물실 이관
.gitignore	올리지 말 파일 목록	대외비 문서 분류

- .gitignore 는 4-6회차에서 중요해진다. 비밀번호·API 키가 담긴 .env 파일을 GitHub에 올리는 것이 가장 흔한 보안 사고
- 브랜치·풀 리퀘스트는 "원본을 건드리지 않고 시험하는 사본" 정도로만 소개하고 넘어간다

③ 말로 하는 Git — 외울 필요 없는 문장들

하고 싶은 것	Claude Code에게 하는 말
시작	이 폴더를 Git 저장소로 만들고, 올리면 안 되는 파일은 .gitignore에 넣어줘
저장	지금까지 바뀐 내용을 한글 메시지로 커밋해줘
확인	커밋 이력을 날짜와 함께 표로 보여줘
비교	마지막 커밋 이후 뭐가 바뀌었는지 쉽게 설명해줘
되돌리기	어제 커밋 상태로 파일을 되돌려줘. 지금 상태는 따로 보관하고
업로드	GitHub에 비공개 저장소를 만들고 올려줘

습관 하나만 남깁니다: **잘 돌아갈 때마다 커밋**. "기능 하나 완성 = 커밋 하나"를 CLAUDE.md에 규칙으로 적어 두면 Claude가 알아서 챙깁니다.

사전 준비 (강의 전 완료)

- 1. GitHub 계정 가입 (업무용 이메일 권장)
- 2. Git 설치 확인: Windows는 Git for Windows, macOS는 처음 git 실행 시 안내에 따라 설치
- 3. GitHub 로그인 연동: Claude Code에게 GitHub CLI(gh)를 설치하고 로그인하는 방법을 안내해줘 라고 요청. 브라우저 로그인은 본인이 직접 완료

실습

- 1. 실습 3-1 첫 커밋: 2회차 폴더에서 이 폴더를 Git 저장소로 만들고 첫 커밋해줘. 실습용 엑셀 원본은 올리지 않게 해줘.
- 2. 실습 3-2 일부러 망치고 되살리기: 출장비 계산기에 배경을 형광 분홍으로, 글자를 3배로 키워줘 → 확인 후 방금 변경을 취소하고 마지막 커밋 상태로 돌려줘 → 원래대로 돌아온 것을 눈으로 확인
- 3. 실습 3-3 GitHub에 올리기: GitHub에 'expense-calculator'라는 비공개 저장소를 만들고 푸시해줘. 끝나면 주소를 알려줘. → 브라우저로 열어 파일과 커밋 이력 확인

정리

- 커밋 = 세이프 포인트, 푸시 = 클라우드 백업. 명령어는 Claude가, 판단은 내가
- 저장소는 기본 비공개(private). 업무 자료가 들어간 저장소를 공개로 만들지 않는다
- 다음 회차: GitHub에 올린 코드를 Vercel에 연결하면 그대로 웹사이트가 된다

강사 노트

- 이 회차의 최대 병목은 GitHub 로그인 연동. 사전 과제로 계정 가입까지는 반드시 완료하게 하고, 조교가 있으면 배치
- 2단계 인증(2FA) 설정 요구가 뜨는 경우가 있으니 휴대폰 준비 안내
- 예상 질문 — "그냥 클라우드 드라이브에 올리면 안 되나": 드라이브는 파일을, Git은 '무엇을 왜 바꿨는지'를 저장한다. 그리고 배포 서비스(Vercel)가 GitHub와 연결된다

4회차 · 웹앱으로 키우고 Vercel로 배포하기 (40분)

이번 회차의 한 문장: "내 PC에만 있던 도구가 링크 하나로 동료에게 간다." GitHub에 올린 코드를 Vercel에 연결해 인터넷 주소를 가진 웹앱으로 만듭니다.

학습 목표

- 로컬 도구와 서버에 올린 도구의 차이를 안다
- Claude Code로 웹앱을 만들고 내 PC에서 띄워 본다 (localhost)
- GitHub → Vercel 연결로 배포하고, 수정이 자동 반영되는 것을 확인한다

진행표

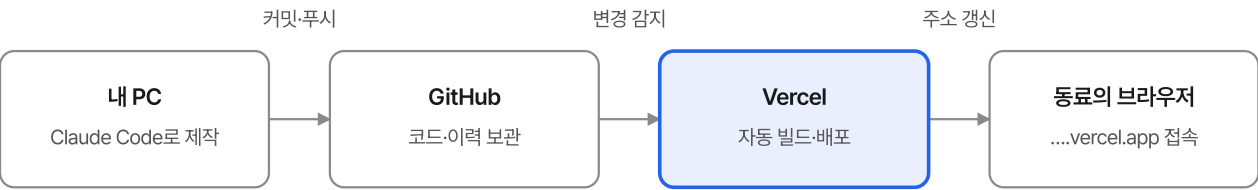
시간	내용	방식
0-8분	로컬 vs 서버, 배포 흐름, Vercel 요금제와 주의점	강의
8-20분	실습 4-1 기획 인터뷰 → 웹앱 제작 → 로컬 실행	실습
20-32분	실습 4-2 GitHub 푸시 → Vercel 배포	따라하기
32-38분	실습 4-3 수정 → 푸시 → 자동 반영, 주소 교환	실습
38-40분	정리	강의

강의 내용

① 로컬 도구에서 서비스로

- 2회차의 HTML 도구는 내 PC에서만 열린다. 동료가 쓰려면 파일을 메일로 보내야 하고, 고칠 때마다 다시 보내야 한다
- 서버에 올리면 주소 하나로 누구나 같은 최신 버전을 쓴다. 이것이 배포(deploy)
- Vercel은 GitHub 저장소를 연결해 두면, **푸시할 때마다 자동으로 다시 배포**한다. 3회차에서 배운 커밋·푸시가 곧 업데이트 버튼이 된다

푸시 한 번이면 동료 화면까지 자동으로 바뀐다



사람이 하는 일은 왼쪽 두 칸뿐, Vercel 이후는 자동 (1~2분)

그림 2. 배포 흐름

② 기술 용어는 딱 세 개만

- Next.js**: 웹앱의 뼈대(프레임워크). Vercel이 만든 것이라 궁합이 좋다. 구조는 Claude가 알아서 잡는다
- localhost**: 내 PC에서만 열리는 시험용 주소. 배포 전 확인용

- **환경변수:** 비밀번호·키처럼 코드에 적으면 안 되는 값을 따로 보관하는 칸. 5-6회차에서 본격 사용

③ Vercel 무료 요금제(Hobby) — 꼭 알아야 할 것

- 무료이고 프로젝트 200개, 하루 배포 100회까지 가능해 교육·시제품에는 충분하다
- 단, **비상업·개인 용도로만** 쓸 수 있다 (Vercel Hobby Plan). 기관의 정식 업무 서비스로 운영하려면 Pro(개발자 1인당 월 20달러) 또는 학교 서버를 검토해야 한다
- 배포 주소는 기본적으로 누구나 열 수 있다. 내부용이면 Vercel Authentication(무료 포함)으로 접근을 제한하거나, 개인정보를 넣지 않는다

사전 준비

1. Node.js LTS 버전 설치 (웹앱 실행에 필요). 또는 Claude Code에게 `Node.js가 설치돼 있는지 확인하고 없으면 설치 방법을 안내해줘`
2. Vercel 가입: "Continue with GitHub"로 가입하면 저장소 연결이 가장 쉽다

실습

주제 예시: **연구과제 공모 일정 D-day** 보드 (사업명, 주관기관, 마감일, 담당자를 카드로 보여주고 마감 임박 순 정렬). 데이터는 일단 코드 안에 가상으로 넣고, 5회차에서 DB로 옮깁니다.

1. **실습 4-1 기획부터 로컬 실행까지** - 연구과제 공모 일정을 팀원들과 공유하는 웹앱을 만들고 싶어. 필요한 기능을 3개 이내로 질문해서 정하고, 기획서.md를 써줘. - 기획서대로 Next.js로 만들고, 내 PC에서 실행해서 브라우저로 볼 수 있게 해줘. 가상 공모 10건을 넣어줘. - localhost 주소를 열어 확인하고, 고칠 점을 말로 요청 ("마감 7일 이내는 빨간색으로")
2. **실습 4-2 배포** - 커밋하고 GitHub 비공개 저장소로 푸시해줘. - Vercel 대시보드 → Add New → Project → 방금 저장소 Import → Deploy. 1-2분 후 `....vercel.app` 주소 발급 - 대안: Claude Code에게 `Vercel CLI로 배포해줘` 라고 해도 된다 (로그인은 본인이 브라우저에서)
3. **실습 4-3 자동 반영 체험** - 제목을 '삼육대 산단 공모 보드'로 바꾸고 커밋·푸시해줘. → 1분 뒤 배포 주소에 반영된 것 확인 - 옆 사람과 주소를 바꿔 휴대폰으로 열어 보기

정리

- 흐름: 만들기(Claude Code) → 저장(Git) → 올리기(GitHub) → 배포(Vercel, 자동)
- 누구나 볼 수 있는 주소다. 개인정보·내부 자료는 올리지 않는다
- 다음 회차: 지금은 공모 데이터를 고치려면 코드를 바꿔야 한다. 화면에서 바로 추가·수정하려면 데이터베이스가 필요하다

강사 노트

- Node.js 설치와 첫 `npm install` 에 시간이 걸린다. 사전 과제로 설치 확인, 또는 강사가 만든 시작 템플릿 저장소에서 출발
- 빌드가 실패하면 Vercel의 에러 로그를 복사해 Claude Code에 붙여 넣고 `이 에러를 고치고 다시 푸시해줘`. 이 장면 자체가 좋은 교재
- 학교 네트워크에서 npm·Vercel 접속이 막히지 않는지 전산팀과 사전 확인
- Hobby 요금제의 비상업 조건은 산학협력단 정식 도입 시 담당 부서와 별도 검토 필요

5회차 · 데이터베이스 붙이기: 접수·신청형 업무 앱 (40분)

이번 회차의 한 문장: "데이터베이스는 여러 사람이 동시에 쓰는 공유 엑셀이다. 단, 문단속을 해야 한다." 4회차 공모 보드에 Supabase를 연결해 화면에서 데이터를 추가·수정하는 앱으로 키웁니다.

학습 목표

- 테이블·행·열을 엑셀에 빗대어 설명할 수 있다
- Supabase 프로젝트를 만들고 웹앱과 연결한다
- 환경변수와 행 수준 보안(RLS)이 왜 필요한지 안다

진행표

시간	내용	방식
0-10분	DB 개념, Supabase vs Neon, 보안 3원칙	강의
10-18분	실습 5-1 Supabase 프로젝트 만들고 테이블 설계	따라하기
18-30분	실습 5-2 공모 보드를 DB로 전환, 등록 폼 추가	실습
30-37분	실습 5-3 보안 설정, Vercel 환경변수, 재배포	실습
37-40분	정리	강의

강의 내용

① 데이터베이스 = 문단속하는 공유 엑셀

엑셀	데이터베이스
시트	테이블 (예: 공모사업 , 행사신청)
열 제목	컬럼 (사업명, 마감일, 담당자)
한 줄	행 (공모 1건)
파일을 메일로 돌리기	웹앱이 한 곳에서 읽고 씀
누구나 아무 칸이나 수정	누가 무엇을 보고 쓸지 규칙으로 제한

② Supabase와 Neon — 무엇을 고를까

항목	Supabase	Neon
정체	DB + 로그인 + 파일 저장 + 관리 화면 묶음	DB(PostgreSQL)에 집중
무료 한도	활성 프로젝트 2개, DB 500MB, 1주일 미사용 시 일시정지	프로젝트 100개, 프로젝트당 1GB, 5분 미사용 시 자동 절전
유료 시작	월 25달러부터	사용량 기반, 월 최소금액 없음
비개발자에게	표 편집 화면이 엑셀처럼 쉬움, 로그인 기능 내장	Vercel 대시보드에서 바로 연결 가능
이 과정의 선택	기본 실습	대안 소개

출처: Supabase 요금제, Neon 요금제 (2026-10 확인)

㉓ 보안 3원칙 — 바이브 코딩 사고의 대부분이 여기서 난다

1. 키는 환경변수에: DB 접속 키를 코드에 직접 적지 않는다. 내 PC는 `.env.local`, 배포본은 Vercel 환경변수 메뉴에 `.env.local` 이 `.gitignore` 에 있는지 확인
2. 두 개의 키를 구분: 화면에 노출돼도 되는 공개 키(anon/publishable)와, 모든 권한을 가진 비밀 키(service_role/secret). 비밀 키는 절대 브라우저 쪽 코드에 넣지 않는다
3. RLS(행 수준 보안)를 켜다: "누구나 읽기 가능, 로그인한 담당자만 쓰기 가능" 같은 규칙. 1회차에서 본 웹앱 170개 개인정보 노출 사고가 바로 이 설정 누락에서 비롯됐다

Claude Code에게는 이렇게 묻습니다: 이 앱의 보안 설정을 점검해줘. 키가 코드에 노출돼 있거나, RLS 없이 누구나 쓸 수 있는 테이블이 있는지 확인하고 목록으로 보여줘.

사전 준비

- Supabase 가입 (GitHub 계정으로). 무료 프로젝트는 2개까지이므로 기존 프로젝트가 있으면 정리
- 리전은 서울(Northeast Asia, Seoul) 선택 권장

실습

1. 실습 5-1 테이블 설계는 AI와 대화로 - Supabase에서 새 프로젝트 생성 (데이터베이스 비밀번호는 본인이 입력하고 따로 보관) - Claude Code에: 공모 보드에 필요한 테이블을 설계해줘. 컨럼과 이유를 표로 보여주고, Supabase SQL Editor에 붙여 넣을 SQL을 만들어줘. RLS 정책도 포함해줘. - SQL을 직접 읽어 보고(이해 안 되는 줄은 질문) SQL Editor에서 실행 → Table Editor에서 엑셀처럼 보이는지 확인
2. 실습 5-2 웹앱과 연결 - Supabase 프로젝트 설정에서 URL과 공개 키를 복사 - `.env.local` 파일을 만들어 둘 테니 내가 키를 넣을게. 그 다음 공모 보드가 코드 안 가상 데이터 대신 Supabase에서 읽어오도록 바꿔줘. 공모 등록 폼도 만들어줘. - 키는 채팅창이 아니라 파일에 직접 붙여 넣는 습관 - 로컬에서 폼으로 1건 등록 → Supabase Table Editor에 행이 생기는지 확인
3. 실습 5-3 보안 점검과 재배포 - 위의 보안 점검 프롬프트 실행, 지적사항 수정 - Vercel 프로젝트 Settings → Environment Variables에 같은 키 등록 → 커밋·푸시 → 배포 주소에서 등록이 되는지 확인

대안 주제: 행사 참가 신청 앱 (신청 폼 + 담당자용 신청자 목록 + CSV 다운로드). 단, 실제 개인정보를 받는 서비스는 개인정보 수집 동의·보관 기간 등 법적 요건이 따르므로 교육에서는 가상 데이터만 사용

한 걸음 더: MCP로 DB까지 말하기 (시연만)

MCP는 Claude Code에 외부 서비스를 연결하는 표준 플러그인입니다. Supabase MCP를 연결하면 Claude가 테이블 생성·조회를 직접 합니다 (Supabase MCP 가이드).

```
claude mcp add --scope project --transport http supabase "https://mcp.supabase.com/mcp"
```

Supabase도 실제 운영 데이터에는 읽기 전용(`read_only=true`)과 단일 프로젝트 제한을 권합니다. 1회차의 "DB 삭제 사고"를 다시 언급하며 권한 승인을 끄지 말 것을 강조합니다.

정리

- DB가 붙으면 "보여주는 페이지"가 "일하는 앱"이 된다

- 키는 환경변수, 비밀 키는 서버에서만, RLS는 반드시. AI에게 보안 점검을 시키는 것도 습관으로
- 다음 회차: 이 앱에 AI를 넣는다

강사 노트

- 40분 안에 모두 끝내기 빠듯하다. 4회차 결과물이 없는 수강생을 위해 "4회차 완성본" 저장소를 준비해 두고 거기서 출발
- Supabase 화면은 영문. 프로젝트 생성 → SQL Editor → Table Editor → 키 복사 위치를 스크린샷 안내서로 나눠 주기
- 무료 프로젝트는 1주일 미사용 시 일시정지되므로, 6회차까지 간격이 길면 수업 전에 다시 켜도록 안내
- 수강생이 키를 채팅창에 붙여 넣었다면, 교육용이라도 수업 후 키를 재발급(rotate)하는 법까지 보여 주면 좋다

6회차 · AI API로 지능형 업무 도구 만들기 + 안전하게 쓰기 (40분)

이번 회차의 한 문장: "Claude Code는 만드는 AI, Claude API는 내 앱 안에서 일하는 AI다." 공고문을 붙여 넣으면 AI가 항목을 뽑아 자동 등록하는 기능을 넣고, 기관에서 안전하게 쓰는 원칙으로 과정을 마무리합니다.

학습 목표

- Claude Code와 Claude API의 역할 차이를 안다
- API 키를 안전하게 발급·보관하고, 비용을 어림잡을 수 있다
- 업무 앱에 요약·추출·분류 기능을 넣는다
- 기관에서 AI 도구를 운영할 때의 보안·거버넌스 기준을 안다

진행표

시간	내용	방식
0-8분	만드는 AI vs 일하는 AI, API 키와 비용	강의
8-25분	실습 6-1 공고문 자동 추출·등록 / 6-2 선택	실습
25-33분	보안·거버넌스: 기관에서 쓸 때의 원칙	강의
33-40분	과정 회고, 다음 단계, 수료	토의

강의 내용

① 만드는 AI와 일하는 AI

구분	Claude Code	Claude API
역할	내가 말하면 프로그램을 만들어 줌	내가 만든 프로그램 안에서 매번 일함
쓰는 사람	나 (개발할 때)	앱을 쓰는 동료 모두
비용	Claude 구독료 (Pro 등)	사용한 만큼 별도 과금 (토큰 단위)
예시	"공모 보드 앱을 만들어줘"	앱 안의 [AI로 자동 입력] 버튼

지능형 기능의 세 가지 기본형만 기억하면 대부분의 업무 아이디어가 나옵니다.

- 요약:** 긴 공문·회의록·보고서를 5줄로
- 추출:** 공고문에서 사업명·마감일·지원자격을 표의 칸으로
- 분류:** 문의 메일을 "협약/정산/기타"로 나누고 답변 초안 작성

② API 키와 비용 감각

- 키 발급: Claude Console에서 선불 크레딧 충전 후 API 키 생성. 월 사용 한도를 먼저 설정
- 키는 5회차와 같은 원칙: `.env.local` 과 Vercel 환경변수에만, 그리고 서버 쪽 코드에서만 호출. 브라우저 코드에 넣으면 누구나 내 키로 과금시킬 수 있다
- 요금은 읽은 양(입력)과 쓴 양(출력)의 토큰 수로 계산한다

모델	성격	입력 / 출력 (100만 토큰당)	공고문 1건 추출 어렵값*
Claude Haiku 4.5	가장 빠르고 저렴, 단순 분류·추출	\$1 / \$5	약 8원
Claude Sonnet 5.5	속도와 성능의 균형, 일반 업무	\$2 / \$10	약 15원
Claude Opus 5.5	복잡한 분석·긴 문서	\$4 / \$20	약 30원

*입력 3,000토큰(공고문 A4 2장 안팎) + 출력 500토큰, 1달러=1,400원 가정의 간이 계산. 가격은 2026-09 기준 Anthropic API 요금이며 변동될 수 있다 (Anthropic 요금표)

실습

- 1. 실습 6-1 공고문 붙여 넣기 → 자동 등록 (필수) - Console에서 발급한 키를 `.env.local` 에 직접 붙여 넣기 (채팅창 금지) - 공모 보드에 '공고문으로 등록' 기능을 추가해줘. 공고문 텍스트를 붙여 넣으면 Claude API로 사업명·주관기관·마감일·지원자격을 뽑아 등록 폼에 채워 주고, 내가 확인한 뒤 저장 버튼을 누르게 해줘. API 키는 서버에서만 쓰고, 모델은 Claude Sonnet 5.5로. - 강사가 준비한 가상 공고문 3개로 시험. 마감일을 잘못 뽑는 경우가 있는지 확인 - 포인트: AI는 초안을 채우고, 저장은 사람이 확인 후. 이 설계가 에이전틱 엔지니어링의 "사람 검증"을 앱에 심은 것 - 배포: Vercel 환경변수에 키 등록 → 커밋·푸시
- 2. 실습 6-2 내 PC용 인스턴트 AI 도구 (선택) - 회의록 폴더에 있는 텍스트 파일들을 하나씩 Claude API로 요약해서, 결정사항과 할 일(담당·기한)을 엑셀 한 장으로 정리하는 프로그램을 만들어줘. - 2회차 인스턴트 소프트웨어에 AI를 더하면 무엇이 달라지는지 체험

보안·거버넌스: 기관에서 쓸 때의 7원칙

- 1. 데이터 등급을 먼저 나눈다. 공개 가능 / 내부용 / 개인정보·민감정보. 마지막 등급은 학교 지침과 담당 부서 협의 없이 AI에 넣지 않는다
- 2. 계정 종류별 데이터 정책을 확인한다. API와 Team-Enterprise 같은 상업용 계정은 기본적으로 모델 학습에 쓰이지 않는다. 개인 Pro 계정은 개인정보 설정에서 학습 허용 여부를 직접 확인·선택해야 한다 (Anthropic 개인정보 센터)
- 3. 키는 비밀번호다. 코드·채팅·메신저에 남기지 않고, 노출되면 즉시 재발급한다
- 4. 사용 한도를 걸어 둔다. Console 월 한도, Vercel-Supabase 요금 알림
- 5. AI 결과는 초안이다. 금액·날짜·수신자처럼 틀리면 안 되는 값은 사람이 확인하고 저장·발송한다
- 6. 외부 문서 속 지시문을 조심한다. 공고문·메일·웹페이지에 "이전 지시를 무시하라" 같은 문장을 숨겨 AI를 조종하는 공격 (프롬프트 인젝션)이 있다. Supabase도 MCP 문서에서 같은 위험을 경고한다
- 7. 만든 사람이 떠나도 남는가를 묻는다. 동료가 매일 쓰는 도구가 되면 기획서·CLAUDE.md·GitHub 저장소를 팀 계정으로 옮기고, 필요하면 전산팀에 이관한다

인스턴트 도구를 계속 운영해도 될까? — 판단표

질문	예 → 인스턴트로 충분	아니오 → 정식 도입 검토
나 혼자 또는 소수만 쓰나?	내 PC·Hobby 배포	사용자 관리·로그인 필요
개인정보가 없나?	그대로 사용	개인정보 처리 절차·보안 심사
멈춰도 업무가 돌아가나?	그대로 사용	장애 대응·유료 요금제
돈이 오가거나 외부에 공개되나?	아니오면 그대로	상업용 요금제·기관 승인

과정 마무리 (7분)

- 6회차 흐름 되짚기: 개념 → 내 PC → 안전망(Git) → 배포 → 데이터 → AI. 각 단계마다 사람의 역할은 **정의하고, 승인하고, 검증하는 것**
- 다음 단계 (자율 학습): 자주 쓰는 요청을 스킴·커스텀 명령으로 저장하기, MCP로 구글 드라이브·노션 등 연결하기, 팀 공용 CLAUDE.md 만들기
- 소회의: "내 업무에서 다음 달 안에 만들 도구 하나"를 한 줄로 적고 공유

강사 노트

- API 크레딧은 개인 카드 결제가 필요하다. 산학협력단에서 교육용 조직 계정을 만들어 키를 나눠 주거나, 강사 키로 시연만 하는 방안을 사전에 결정
- 교육용 키를 나눠 주었다면 수업 직후 폐기(revoke)하고, 키마다 사용 한도를 낮게 설정
- 보안 파트는 "하지 말라"가 아니라 "이렇게 하면 쓸 수 있다"로 전달. 학교 자체 AI 활용 지침이 있으면 해당 조항을 슬라이드에 인용

부록

A. 강사 준비 체크리스트

- ☐ 산학협력단과 협의: Claude 계정(개인 Pro vs 기관 Team), API 크레딧 부담 주체, 실습 PC 제공 여부
- ☐ 전산팀 확인: 실습 PC에서 Claude·GitHub·npm·Vercel·Supabase 접속 가능 여부, 프로그램 설치 권한
- ☐ 실습 PC 1대로 전 회차 리허설 (특히 2회차 Python, 4회차 Node.js)
- ☐ 가상 실습 데이터 제작: 뒤섞인 폴더, 집행내역 엑셀 10개(양식 일부러 다르게), 증빙 파일, 공고문 3개, 회의록 5개
- ☐ 회차별 완성본 GitHub 저장소 준비 (뒤쳐진 수강생이 다음 회차에 합류할 수 있도록)
- ☐ 회차별 사전 과제 공지: 3회차 전 GitHub, 4회차 전 Node.js·Vercel, 5회차 전 Supabase
- ☐ 시연용 백업 녹화: 네트워크 문제로 라이브 시연이 안 될 때를 대비
- ☐ 교육용 API 키 발급과 수업 후 폐기 일정

B. 바로 쓰는 업무 프롬프트 모음

상황	프롬프트
폴더 파악	이 폴더 파일들을 종류별로 표로 정리하고 각각 무슨 내용인지 한 줄씩 알려줘
기획부터	나는 [만들고 싶은 것]을 만들고 싶어. 내가 못 떠올린 부분까지 질문해서 기획서.md로 정리해줘
엑셀 취합	[폴더]의 엑셀을 [기준]별로 합쳐 결과 폴더에 저장하고, 원본 합계와 대조한 검산 결과를 보여줘
일괄 변경	[파일들]을 [규칙]으로 바꾸고 싶어. 변경 전/후 표를 먼저 보여주고 승인하면 실행해줘
문서 초안	@[자료파일]을 바탕으로 [문서 종류] 초안을 써줘. 근거가 된 부분을 표시해줘
오류 해결	이 에러가 났어: [붙여넣기]. 원인을 쉽게 설명하고 고친 뒤 다시 실행해서 확인해줘
저장	지금까지 바뀐 내용을 한글 메시지로 커밋해줘
되돌리기	마지막 커밋 상태로 돌려줘. 지금 상태는 따로 보관하고
보안 점검	키가 코드에 노출됐거나, 누구나 쓸 수 있는 데이터가 있는지 점검하고 목록으로 보여줘
설명 듣기	방금 만든 것이 어떻게 동작하는지 비개발자가 알아듣게 5줄로 설명해줘

C. 리서치 근거와 참고자료

확인일 2026-10-06. 요금·무료 한도는 자주 바뀌므로 강의 직전에 다시 확인하세요.

자료	활용 회차	핵심 내용
Karpathy, "vibe coding" 원문 (X, 2025-02-02)	1	바이브 코딩 용어의 기원
Wikipedia: Vibe coding	1	콜린스 올해의 단어 2025, 보안 사고 사례
Karpathy의 Agentic Engineering 요약 (AI Builder Club)	1	"바이브는 바닥을, 에이전틱은 천장을" (Sequoia Ascent 2026)
a16z, Disposable Software (2025-08)	1, 2	인스턴트·일회용 소프트웨어 개념

자료	활용 회차	핵심 내용
How Anthropic teams use Claude Code	1	법무·마케팅 등 비개발 부서 활용 사례
Claude Code 설치 문서	1	설치 명령, 요구 사양, 필요 요금제
Best practices for Claude Code	2, 3	탐색→계획→실행, 검증, CLAUDE.md, 되돌리기
Vercel Hobby Plan	4	무료 한도, 비상업 조건
Supabase 요금제	5	무료 프로젝트 2개, 500MB, 1주 미사용 시 일시정지
Neon 요금제	5	무료 프로젝트당 1GB, 5분 자동 절전
Supabase MCP 가이드	5, 6	Claude Code 연결 명령, 읽기 전용·프롬프트 인젝션 경고
Anthropic 요금표	6	모델별 API 요금

한 문장 요약 — 리서치가 교안에 끼친 영향: 2026년 현재 업계의 흐름은 "아무나 만들 수 있다(바이브 코딩)"에서 "제대로 만들려면 사람이 정의·검증해야 한다(에이전틱 엔지니어링)"로 옮겨 가고 있어, 교안 전체를 "기획·승인·검증은 직원의 강점"이라는 메시지로 묶었습니다.